Class 11: Protein Structure Prediction with AlphaFold

Alexia (A17297003)

Table of contents

Background

Proteins are natures robots responsible for nearly every task in living organisms. Just as robots are programmed to perform specific tasks, proteins, through their distinct 3D shapes (known as their atomic structures), carry out diverse functions from catalysis, signaling and transport to replication, division and movement.

Generating your own structure predictions - ColabFold

For this lab we will use the HIV-Pr Dimer sequence: >HIV-Pr-Dimer PQITLWQR-PLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD QILIE-ICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF:PQITLWQRPLVTIKIGGQLK EALLDT-GADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPT PVNIIGRNLLTQIGCTLNF

We will use AlphaFold2_mmseqs2 Colab notebook to generate a prediction of the 3D structure/model of the amino acid sequence (HIV-PR-Dimer).

Interpreting Results

Once the zip file has been downloaded we will uncompress it to begin result interpretation. Inside the resulting folder/directory we will have a number of .txt, .json, and .pdb files.

Visualization of the models and their estimated reliability

We can use Mol* for visualization of the model PDB files.

Key Note: The B-factor column of each AlphaFold produced PDB file contains the per-residue pLDDT score (a measure of the estimated reliability, or confidence score per-residue) Values above 70 are considered high confidence whereas those scored below 50 are low confidence and hence unreliable.

Custom analysis of resulting models

In this section we will read the results of the more complicated HIV-Pr dimer AlphaFold2 models into R with the help of the Bio3D package.For tidiness we can move our AlphaFold results directory into our RStudio project directory.

```
results_dir <- "hivprdimer_23119"
```

```
pdb_files <- list.files(path=results_dir,
                        pattern="*.pdb",
                        full.names = TRUE)
basename(pdb_files)
```

```
[1] "hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb"
[2] "hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000.pdb"
[3] "hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000.pdb"
[4] "hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000.pdb"
[5] "hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb"
```

```
library(bio3d)
pdbs <- pdbaln(pdb_files, fit=TRUE, exefile="msa")
```

```
Reading PDB files:
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000
```

```
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000
hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000
.....

Extracting sequences

pdb/seq: 1   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_001_alphafold2_multimer_v
pdb/seq: 2   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_002_alphafold2_multimer_v
pdb/seq: 3   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_003_alphafold2_multimer_v
pdb/seq: 4   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_004_alphafold2_multimer_v
pdb/seq: 5   name: hivprdimer_23119/hivprdimer_23119_unrelaxed_rank_005_alphafold2_multimer_v
```

A quick view of model sequences:

```
pdbs
```

```
                              1        .         .         .         .         50
[Truncated_Name:1]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:2]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:3]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:4]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:5]hivprdimer  PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
                              **************************************************
                              1        .         .         .         .         50

                             51        .         .         .         .         100
[Truncated_Name:1]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:2]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:3]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:4]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:5]hivprdimer  GGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
                              **************************************************
                             51        .         .         .         .         100

                            101        .         .         .         .         150
[Truncated_Name:1]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:2]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:3]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:4]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:5]hivprdimer  QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
                              **************************************************
                            101        .         .         .         .         150
```

```
                             151        .         .         .         .       198
[Truncated_Name:1]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:2]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:3]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:4]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:5]hivprdimer   GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
                               ************************************************
                             151        .         .         .         .       198

Call:
  pdbaln(files = pdb_files, fit = TRUE, exefile = "msa")

Class:
  pdbs, fasta

Alignment dimensions:
  5 sequence rows; 198 position columns (198 non-gap, 0 gap)

+ attr: xyz, resno, b, chain, id, ali, resid, sse, call
```

RMSD is a standard measure of structural distance between coordinate sets. We can use the `rmsd()` function to calculate the RMSD between all pairs models.

```
rd <- rmsd(pdbs, fit=T)
```

```
Warning in rmsd(pdbs, fit = T): No indices provided, using the 198 non NA positions
```

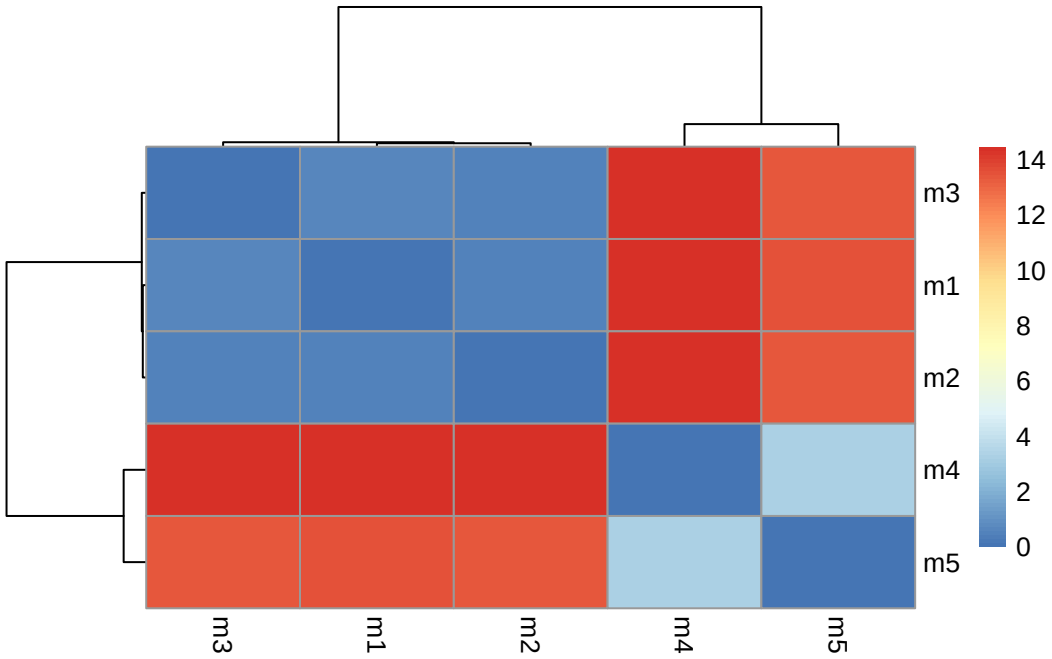
```
range(rd)
```

```
[1]  0.000 14.457
```

Draw a heatmap of these RMSD matrix values:

```
library(pheatmap)

colnames(rd) <- paste0("m",1:5)
rownames(rd) <- paste0("m",1:5)
pheatmap(rd)
```



Here we can see that models 1, 2, and 3 are almost identical to each other while models 4 and 5 share some similarities but are not fully identical in turn making them not as similar as models 1-3. We will see this trend again in the pLDDT and PAE plots further below.

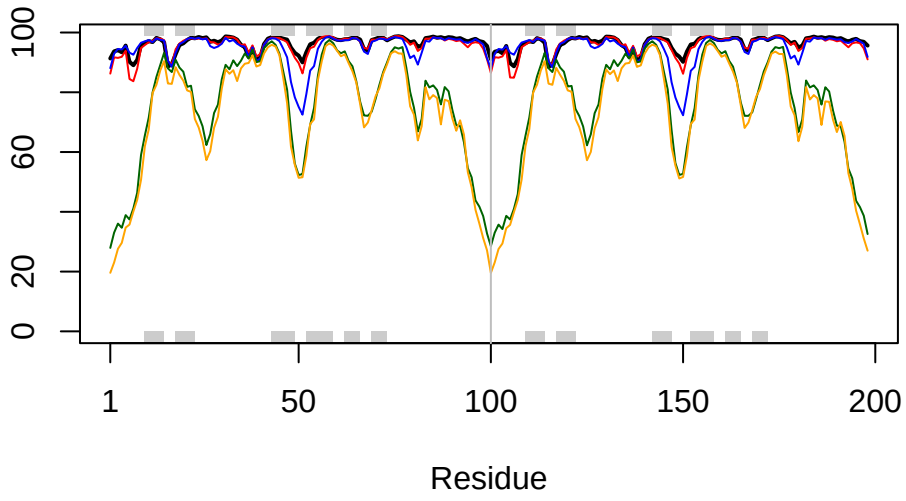
Now lets plot the pLDDT values across all models. Recall that this information is in the B-factor column of each model and that this is stored in our aligned pdbs object as pdbs$b with a row per structure/model.

```
pdb <- read.pdb("1hsg")
```

```
  Note: Accessing on-line PDB file
```

You could optionally obtain secondary structure from a call to stride() or dssp() on any of the model structures.

```
plotb3(pdbs$b[1,], typ="l", lwd=2, sse=pdb)
points(pdbs$b[2,], typ="l", col="red")
points(pdbs$b[3,], typ="l", col="blue")
points(pdbs$b[4,], typ="l", col="darkgreen")
points(pdbs$b[5,], typ="l", col="orange")
abline(v=100, col="gray")
```



We can improve the superposition/fitting of our models by finding the most consistent "rigid core" common across all the models. For this we will use the `core.find()` function:

```
core <- core.find(pdbs)
```

```
 core size 197 of 198  vol = 5754.474
 core size 196 of 198  vol = 4572.526
 core size 195 of 198  vol = 2115.143
 core size 194 of 198  vol = 1525.93
 core size 193 of 198  vol = 1390.829
 core size 192 of 198  vol = 1280.595
 core size 191 of 198  vol = 1171.652
 core size 190 of 198  vol = 1082.83
 core size 189 of 198  vol = 1047.706
 core size 188 of 198  vol = 1021.701
 core size 187 of 198  vol = 1001.511
 core size 186 of 198  vol = 982.408
 core size 185 of 198  vol = 965.11
 core size 184 of 198  vol = 945.291
 core size 183 of 198  vol = 922.93
 core size 182 of 198  vol = 879.676
 core size 181 of 198  vol = 856.189
 core size 180 of 198  vol = 811.173
```

```
core size 179 of 198  vol = 789.408
core size 178 of 198  vol = 764.613
core size 177 of 198  vol = 743.895
core size 176 of 198  vol = 720.032
core size 175 of 198  vol = 699.404
core size 174 of 198  vol = 679.546
core size 173 of 198  vol = 662.076
core size 172 of 198  vol = 640.751
core size 171 of 198  vol = 619.091
core size 170 of 198  vol = 600.089
core size 169 of 198  vol = 582.484
core size 168 of 198  vol = 560.512
core size 167 of 198  vol = 526.05
core size 166 of 198  vol = 494.758
core size 165 of 198  vol = 477.336
core size 164 of 198  vol = 459.201
core size 163 of 198  vol = 444.277
core size 162 of 198  vol = 428.758
core size 161 of 198  vol = 415.985
core size 160 of 198  vol = 401.762
core size 159 of 198  vol = 385.62
core size 158 of 198  vol = 371.755
core size 157 of 198  vol = 357.766
core size 156 of 198  vol = 345.196
core size 155 of 198  vol = 331.481
core size 154 of 198  vol = 317.372
core size 153 of 198  vol = 303.658
core size 152 of 198  vol = 291.234
core size 151 of 198  vol = 276.942
core size 150 of 198  vol = 265.069
core size 149 of 198  vol = 254.695
core size 148 of 198  vol = 244.929
core size 147 of 198  vol = 234.256
core size 146 of 198  vol = 223.932
core size 145 of 198  vol = 214.812
core size 144 of 198  vol = 204.565
core size 143 of 198  vol = 196.615
core size 142 of 198  vol = 182.849
core size 141 of 198  vol = 173.456
core size 140 of 198  vol = 166.316
core size 139 of 198  vol = 157.971
core size 138 of 198  vol = 151.565
core size 137 of 198  vol = 145.31
```

```
core size 136 of 198  vol = 139.486
core size 135 of 198  vol = 133.291
core size 134 of 198  vol = 127.405
core size 133 of 198  vol = 121.952
core size 132 of 198  vol = 116.445
core size 131 of 198  vol = 108.879
core size 130 of 198  vol = 102.219
core size 129 of 198  vol = 94.88
core size 128 of 198  vol = 88.654
core size 127 of 198  vol = 83.357
core size 126 of 198  vol = 79.603
core size 125 of 198  vol = 77.313
core size 124 of 198  vol = 74.084
core size 123 of 198  vol = 70.385
core size 122 of 198  vol = 66.687
core size 121 of 198  vol = 62.458
core size 120 of 198  vol = 59.279
core size 119 of 198  vol = 56.355
core size 118 of 198  vol = 53.213
core size 117 of 198  vol = 49.293
core size 116 of 198  vol = 46.5
core size 115 of 198  vol = 42.868
core size 114 of 198  vol = 39.993
core size 113 of 198  vol = 36.701
core size 112 of 198  vol = 34.827
core size 111 of 198  vol = 32.668
core size 110 of 198  vol = 29.561
core size 109 of 198  vol = 26.953
core size 108 of 198  vol = 24.475
core size 107 of 198  vol = 21.998
core size 106 of 198  vol = 20.512
core size 105 of 198  vol = 18.384
core size 104 of 198  vol = 16.871
core size 103 of 198  vol = 14.911
core size 102 of 198  vol = 13.128
core size 101 of 198  vol = 11.113
core size 100 of 198  vol = 9.381
core size 99 of 198  vol = 7.991
core size 98 of 198  vol = 6.99
core size 97 of 198  vol = 6.059
core size 96 of 198  vol = 5.128
core size 95 of 198  vol = 4.231
core size 94 of 198  vol = 3.692
```

```
 core size 93 of 198  vol = 3.126
 core size 92 of 198  vol = 2.754
 core size 91 of 198  vol = 2.271
 core size 90 of 198  vol = 1.642
 core size 89 of 198  vol = 1.065
 core size 88 of 198  vol = 0.644
 core size 87 of 198  vol = 0.444
 FINISHED: Min vol ( 0.5 ) reached
```

We can now use the identified core atom positions as a basis for a more suitable superposition and write out the fitted structures to a directory called corefit_structures:

```
core.inds <- print(core, vol=0.5)
```

```
# 88 positions (cumulative volume <= 0.5 Angstrom^3)
  start end length
1     8  95     88
```

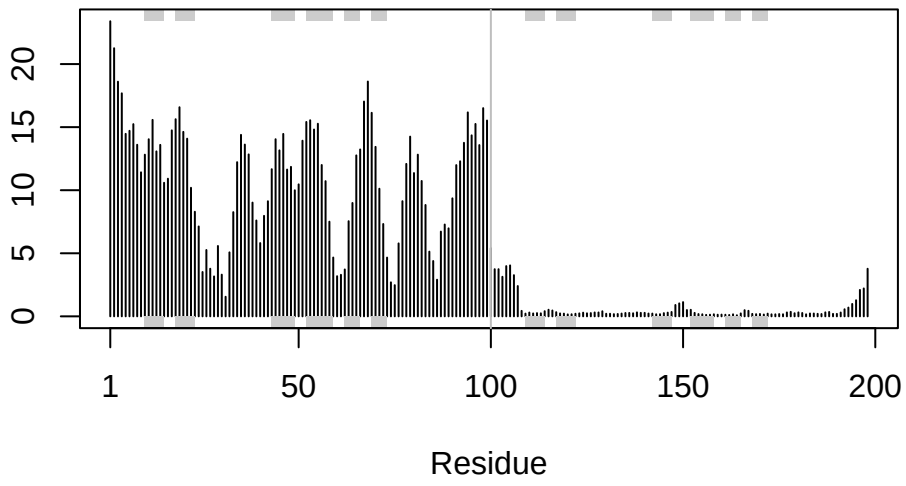
```
xyz <- pdbfit(pdbs, core.inds, outpath="corefit_structures")
```

The resulting superposed coordinates are written to a new director called corefit_structures/. We can now open these in Mol* and color by the Atom Property of Uncertainty/Disorder.

Now we can examine the RMSF between positions of the structure. RMSF is an often used measure of conformational variance along the structure:

```
rf <- rmsf(xyz)

plotb3(rf, sse=pdb)
abline(v=100, col="gray", ylab="RMSF")
```



Here we see that the first chain is much more variable than the second chain. The second chain is very similar across the different models - we saw this in Mol* previously.

Predicted Alignment Error for domains

Independent of the 3D structure, AlphaFold produces an output called Predicted Aligned Error (PAE). This is detailed in the JSON format result files, one for each model structure.

```
library(jsonlite)

pae_files <- list.files(path=results_dir,
                        pattern=".*model.*\\.json",
                        full.names = TRUE)
```

For example purposes lets read the 1st and 5th files (you can read the others and make similar plots).

```
pae1 <- read_json(pae_files[1],simplifyVector = TRUE)
pae5 <- read_json(pae_files[5],simplifyVector = TRUE)

attributes(pae1)
```

```
$names
[1] "plddt"   "max_pae" "pae"     "ptm"     "iptm"
```

```
head(pae1$plddt)
```

```
[1] 91.31 93.69 94.12 93.38 95.62 90.19
```

The maximum PAE values are useful for ranking models. Here we can see that model 5 is much worse than model 6. The lower the PAE score the better. How about the other models, what are their max PAE scores?

```
pae1$max_pae
```

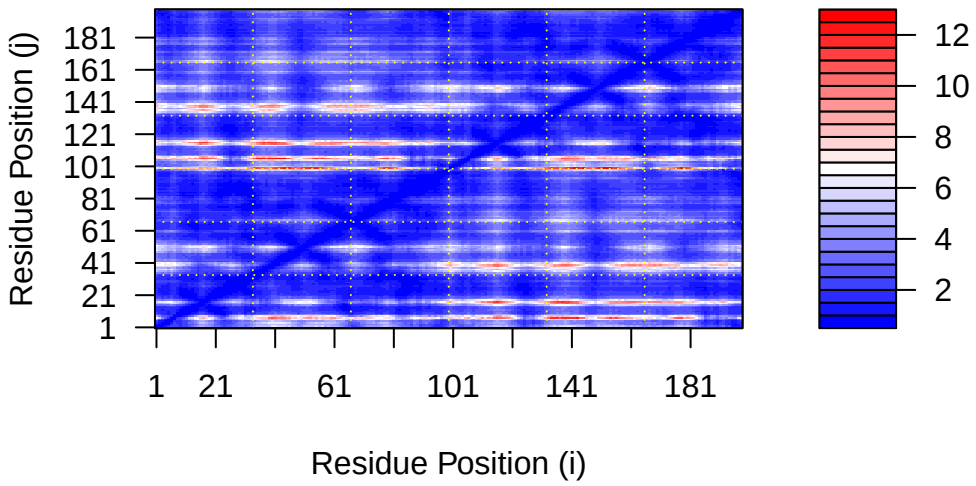
```
[1] 12.57812
```

```
pae5$max_pae
```

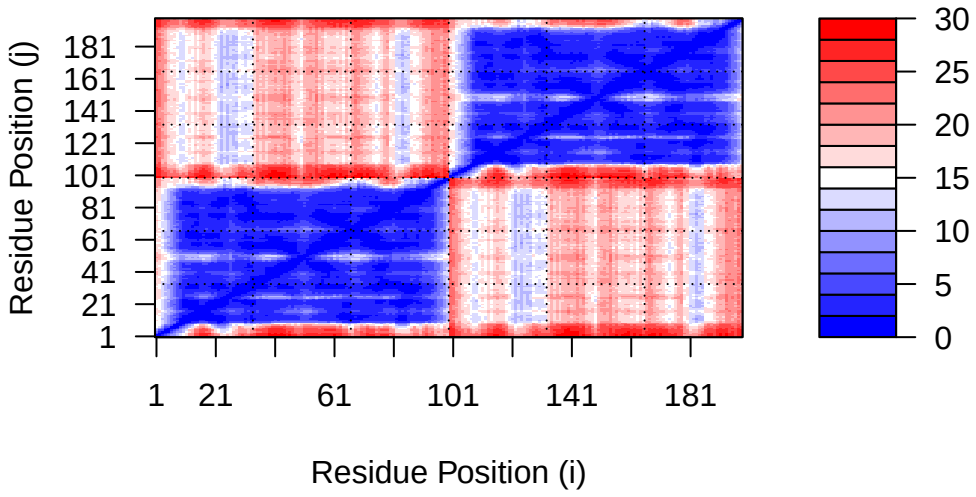
```
[1] 29.82812
```

We can plot the N by N (where N is the number of residues) PAE scores with ggplot or with functions from the Bio3D package:

```
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)")
```

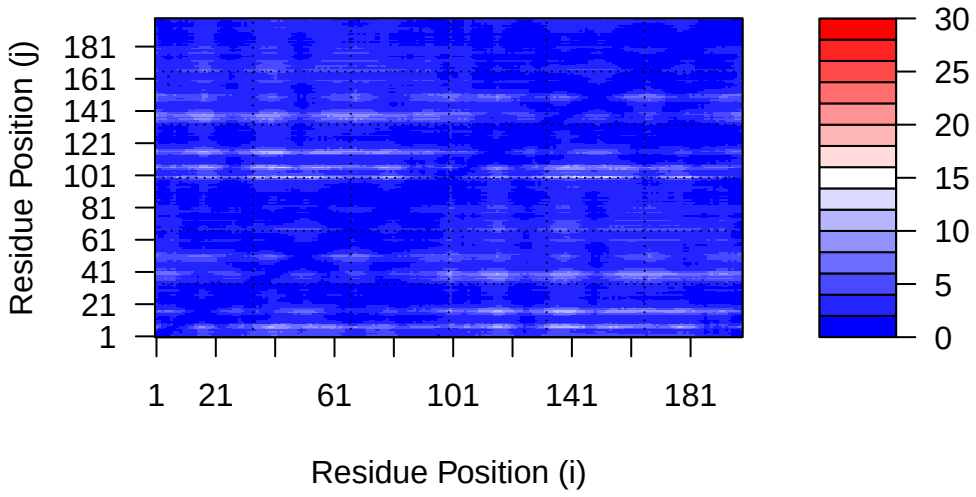



```
plot.dmat(pae5$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



We should really plot all of these using the same z range. Here is the model 1 plot again but this time using the same data range as the plot for model 5:

```
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



Residue conservation from alignment file

```
aln_file <- list.files(path=results_dir,
                       pattern=".a3m$",
                        full.names = TRUE)
aln_file
```

```
[1] "hivprdimer_23119/hivprdimer_23119.a3m"
```

```
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
[1] " ** Duplicated sequence id's: 101 **"
[2] " ** Duplicated sequence id's: 101 **"
```

How many sequences are in this alignment?

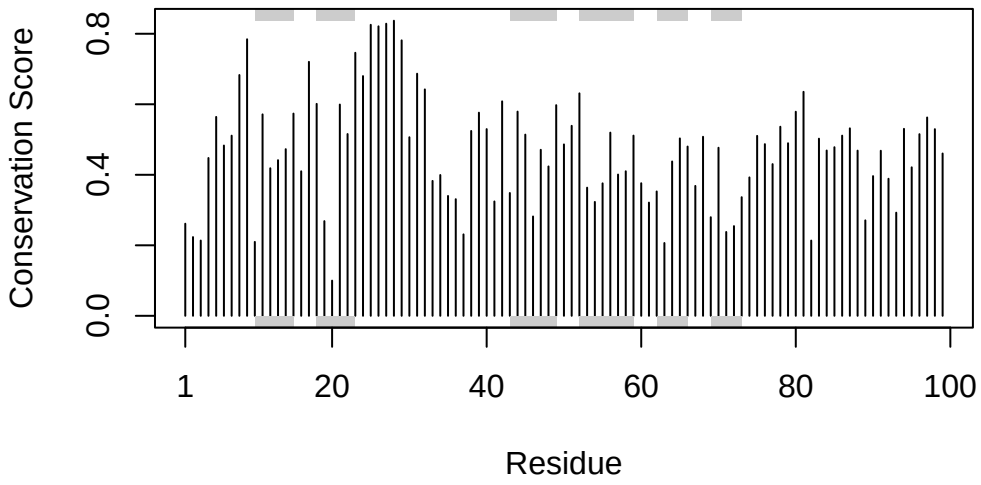
```
dim(aln$ali)
```

```
[1] 5397  132
```

We can score residue conservation in the alignment with the `conserv()` function.

```
sim <- conserv(aln)
```

```
plotb3(sim[1:99], sse=trim.pdb(pdb, chain="A"),
       ylab="Conservation Score")
```



Note the conserved Active Site residues D25, T26, G27, A28. These positions will stand out if we generate a consensus sequence with a high cutoff value:

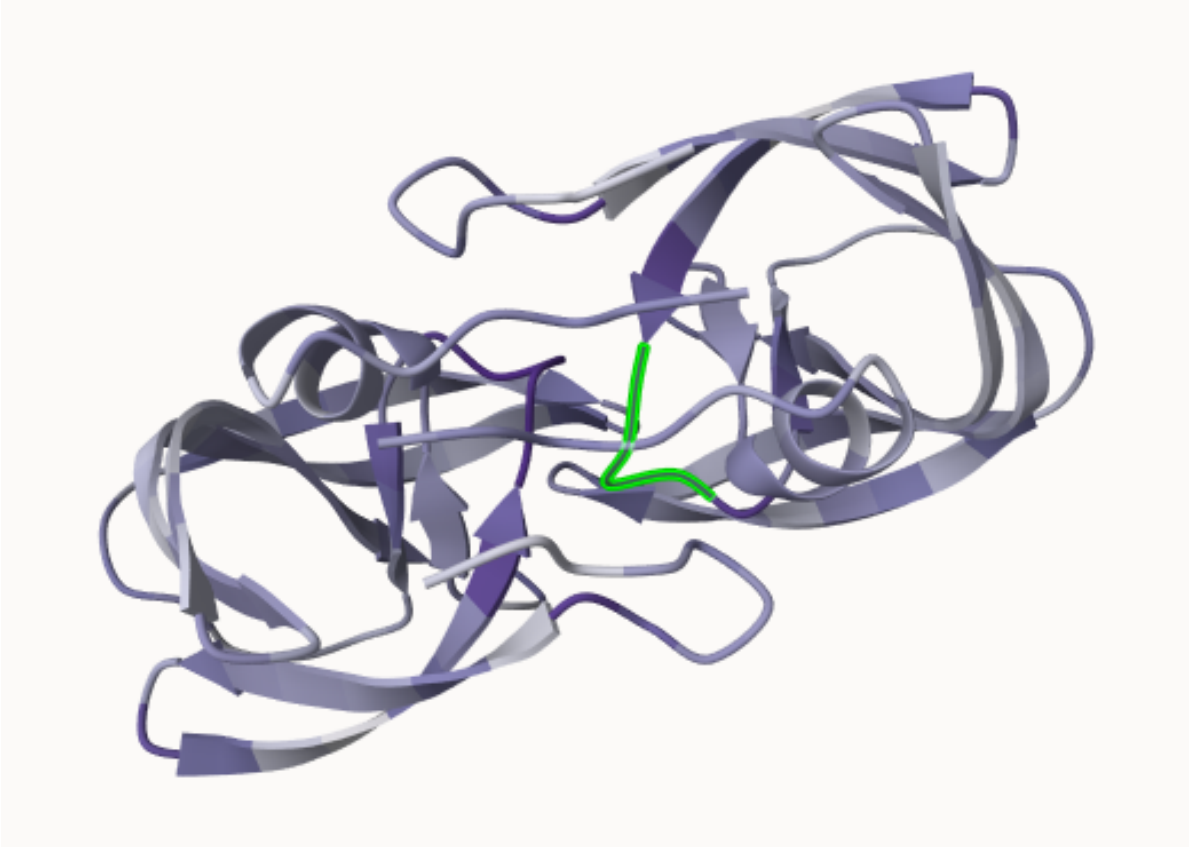
```
con <- consensus(aln, cutoff = 0.9)
con$seq
```

```
  [1] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [19] "-" "-" "-" "-" "-" "-" "D" "T" "G" "A" "-" "-" "-" "-" "-" "-" "-" "-"
 [37] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [55] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [73] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
 [91] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[109] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[127] "-" "-" "-" "-" "-" "-"
```

For a final visualization of these functionally important sites we can map this conservation score to the Occupancy column of a PDB file for viewing in molecular viewer programs such as Mol*, PyMol, VMD, chimera etc.

```
m1.pdb <- read.pdb(pdb_files[1])
occ <- vec2resno(c(sim[1:99], sim[1:99]), m1.pdb$atom$resno)
write.pdb(m1.pdb, o=occ, file="m1_conserv.pdb")
```

Here is an image of this data generated from and Mol* using coloring by Occupancy. This is done in a similar manor to the pLDDT coloring procedure detailed above



Note that we can now clearly see the central conserved active site in this model where the natural peptide substrate (and small molecule inhibitors) would bind between domains. The DTGA motif of one chain is highlighted in green